

CONSTRUIR UN **CONTRATO** PASO A PASO, ROMPIÉNDOLO SEIS VECES

Seis versiones del mismo contrato. Las seis compilan. **Las cinco primeras están mal**, y cada una está mal de una forma distinta.

Ninguna función aparece porque el temario lo diga: cada una existe porque la versión anterior se rompió sin ella.

DÓNDE	QUÉ HACE FALTA	CUÁNTO TOMA SOLOS	DINERO REAL
Remix app.remix.live	Un navegador. Nada instalado.	2 h 30 escribiendo	Ninguno. Nunca.

CÓMO SE USA ESTA GUÍA

En clase: no corran detrás copiando. Miren, pregunten, y marquen en esta guía la comprobación que no les quedó clara. Todo lo que el profesor escriba en vivo está acá.

En casa: háganlo **de nuevo desde cero**, con el archivo vacío, **escribiendo**. Copiar y pegar los seis pasos toma cuatro minutos y no sirve para nada. Escribirlos toma dos horas y media y es la diferencia entre haber visto Solidity y saber Solidity.

Los seis pasos son el MISMO archivo que crece. No se crean seis archivos. Cada paso agrega o cambia unas líneas sobre el anterior. El código completo de cada versión está en `material/clase-solidity-paso-a-paso/pasos/paso-1.sol` ... `paso-6.sol` : ábranlo **después** de intentarlo, no antes.

Si algo no coincide con lo que dice la guía, vayan a la sección 9 (errores frecuentes). La interfaz de Remix cambia de vez en cuando: si un botón no está exactamente donde dice acá, busquen el **nombre de la acción**; la lógica no cambia.

SEGURIDAD · ESTO NO SE NEGOCIA NUNCA

- Todo este ejercicio se hace en la **Remix VM** o en una **red local**. Nunca en una red real, nunca con dinero real.
- Los 100 ETH de cada cuenta de la Remix VM **no existen**. Son de juguete y solo valen dentro de esa pestaña.
- La frase de recuperación de una billetera no se escribe en Remix, ni en un archivo, ni en un chat, ni se le muestra al profesor. Nadie legítimo la pide.
- Este contrato es **material de clase sin auditar**. No se despliega en una red real ni se usa con plata de nadie.
- En la cadena no va ningún dato personal: ni nombres, ni cédulas, ni correos. En este ejercicio solo van direcciones y nombres de equipos.

01 El problema, y qué van a terminar teniendo

LEER PRIMERO

El laboratorio presta equipos: proyectores, kits de Arduino, cámaras. El registro vive en una hoja de papel pegada a la pared. Nadie sabe quién tiene el proyector, nadie sabe desde cuándo, y cuando no vuelve, no pasa nada.

Al final de los seis pasos, el contrato hace esto:

FUNCIÓN	QUÉ HACE	QUIÉN LA LLAMA
<code>registrar(nombre)</code>	Agrega un equipo al inventario y devuelve su <code>id</code> . Emite un evento.	Solo el encargado
<code>prestar(id, dias)</code>	Se lleva el equipo por un plazo de 1 a 14 días, dejando el depósito exacto.	Cualquiera
<code>devolver(id)</code>	Devuelve el equipo. A tiempo: el depósito le queda acreditado. En mora: queda para el laboratorio.	Solo quien lo tiene
<code>retirar()</code>	Cobra lo que el contrato le debe a quien llama.	Cualquiera con saldo
<code>recuperar(id)</code>	Recupera un equipo a la fuerza y se queda con el depósito.	Solo el encargado
<code>estaDisponible(id)</code> <code>quienLoTiene(id)</code> <code>estaVencido(id)</code> <code>diasDeMora(id)</code>	Consultas. Son <code>view</code> : leer no cuesta nada y no genera una transacción.	Cualquiera, gratis

LA IDEA QUE ATRAVIESA TODA LA GUÍA

Cada paso **rompe** la versión anterior en vivo, y la función siguiente existe **para arreglar esa rotura concreta**. Cuando lleguen al paso 6, deberían poder explicar cada palabra del contrato respondiendo «¿qué se rompía sin esto?». Si hay una que no pueden justificar así, pregúntenla.

Y atención: la rotura del paso 1 **no es un error rojo**. Es una transacción que sale perfecta y produce algo imposible. Esa es la clase de error que de verdad cuesta plata.

0

UNA SOLA VEZ, AL EMPEZAR

Entorno completo dentro del navegador

QUÉ HACER

CÓMO SÉ QUE QUEDÓ

- | | | |
|---|--|--|
| 1 | <p>Abran <code>https://app.remix.live</code>. No pide cuenta ni instalar nada.</p> <p>Remix se mudó de dirección en septiembre de 2026. La antigua, <code>remix.ethereum.org</code>, todavía abre, pero la oficial es esta: actualicen el marcador. Si alguna vez ven otra dirección parecida, desconfíen.</p> | <p>A la izquierda hay una barra de íconos: cada uno es un complemento. Hoy usan tres: el explorador de archivos, el compilador de Solidity y Deploy & Run Transactions. Abajo, la terminal.</p> |
| 2 | <p>En el explorador de archivos, creen un archivo nuevo llamado <code>PrestamoEquipos.sol</code>.</p> | <p>Aparece en la lista y se abre vacío en el editor. <code>.sol</code> es la extensión por omisión en Remix.</p> |
| 3 | <p>Abran el compilador de Solidity y elijan la versión <code>0.8.34+commit.80d5c536</code>.</p> | <p>Hoy Remix abre con esa misma versión, así que lo normal es que ya esté puesta: compruébenlo igual. Es la versión congelada del curso y con la que están verificados los seis pasos. Si algún día Remix abre con otra, se cambia aquí, nunca en el <code>pragma</code> del archivo.</p> |
| 4 | <p>Abran Deploy & Run Transactions. En ENVIRONMENT, elijan la primera opción Remix VM del desplegable.</p> | <p>El selector <i>Account</i> lista 10 cuentas con 100 ETH cada una. Si no las ven, no están en la Remix VM. (Según la versión, el <i>hard fork</i> aparece entre paréntesis en cada Remix VM o en un segundo desplegable al lado, con un nombre como <i>Osaka</i>. Cualquiera sirve para esta clase.)</p> |

EL CICLO QUE VAN A REPETIR SEIS VECES

escribir → **compilar** → **Deploy** → **probar a mano con la tabla del paso**. Y una advertencia que ahorra media hora de confusión: **cada Deploy crea un contrato nuevo y vacío**. El anterior sigue en la lista con su estado. Usen siempre **el último** de *Deployed Contracts*, y expándalo con el triángulito de la izquierda para ver sus funciones.

Los botones vienen en colores según lo que hace la función: los de **lectura** (`view`) son de un color y no cuestan nada; los que hacen una **transacción**, de otro; y los que además aceptan ether (`payable`), de un tercero. El color exacto depende del tema visual de Remix. Lo que importa: unos son gratis y otros no.

1

QUÉ HAY QUE ESCRIBIR · ARCHIVO COMPLETO EN `pasos/paso-1.sol`

15 líneas: dos variables de estado y dos funciones

- La licencia (`// SPDX-License-Identifier: MIT`) y `pragma solidity 0.8.34;`.
- `contract PrestamoEquipos { ... }.`
- Dos variables de **estado**: `string public nombreEquipo = "Proyector Epson";` y `address public prestadoA;`.
- `function prestar() public`, que pone `prestadoA = msg.sender;`.
- `function devolver() public`, que pone `prestadoA = address(0);`.

EL POR QUÉ DE CADA COSA, EN UNA LÍNEA

Variable de estado: vive en la cadena; sigue ahí mañana y después de cerrar el navegador (en una cadena real).

`public` en una **variable**: le regala a usted la función de lectura, gratis; no hay que escribirla.

`msg.sender`: la dirección que **firmó** esta transacción. En Solidity nadie dice quién es: lo demuestra firmando. No hay campo de usuario que se pueda mentir.

`address(0)`: la dirección cero, el «vacío» de Ethereum. La usamos para decir «no lo tiene nadie». Va a volver todo el día.

Qué hay que comprobar

CUENTA	QUÉ LLAMAR	SALIDA ESPERADA
1	Compilar.	Marca verde en el ícono del compilador y cero avisos . Si hay avisos, algo está distinto del archivo del paso.
2	Cuenta 1 → Deploy , y leer <code>prestadoA</code> .	<code>0x00</code> . No hubo que inicializar nada: en Solidity todo arranca en cero.
3	Leer <code>nombreEquipo</code> .	<code>Proyector Epson</code> .
4	Cuenta 1 → <code>prestar()</code> , y volver a leer <code>prestadoA</code> .	La dirección de la cuenta 1. Compárenla con el selector Account: los primeros cuatro caracteres y los últimos cuatro.
5	Cambiar <i>Account</i> a la cuenta 2 → <code>prestar()</code> otra vez → leer <code>prestadoA</code> .	Funciona, y queda la cuenta 2. La transacción sale bien, sin un solo error.

QUÉ SE ROMPIÓ · Y POR QUÉ ESTA ES LA ROTURA MÁS IMPORTANTE DE LA GUÍA

El equipo cambió de manos y la cuenta 1 **nunca lo devolvió**. No hay ningún mensaje de error, porque el programa hizo exactamente lo que se le pidió.

La rotura no es un error rojo: es que **todo salió bien y el resultado es imposible** en la vida real. La mayoría de los errores graves en contratos son así: no son excepciones, son reglas que nadie escribió.

Pregúntense antes de seguir: ¿qué tendría que haber pasado cuando la cuenta 2 apretó el botón? → Que la transacción se cayera. Eso es el paso 2.

2

QUÉ HAY QUE ESCRIBIR · `pasos/paso-2.sol`

Sobre el mismo archivo: dos errores, una consulta y dos guardias

- Dos declaraciones nuevas, dentro del contrato: `error EquipoOcupado(address loTiene);` y `error NoLoTienesTu();`.
- `function estaDisponible() public view returns (bool)`, que devuelve `prestadoA == address(0)`.
- Primera línea de `prestar()`: `if (!estaDisponible()) revert EquipoOcupado(prestadoA);`.
- Primera línea de `devolver()`: `if (prestadoA != msg.sender) revert NoLoTienesTu();`.

EL POR QUÉ DE CADA COSA, EN UNA LÍNEA

Error personalizado: el «no» del contrato con nombre propio y con datos adentro.

`EquipoOcupado(prestadoA)` no dice «no se puede»: dice *quién lo tiene*. Cuesta menos gas que un texto y una interfaz lo puede distinguir y mostrar bien.

revert: deshace **todo** lo que la transacción tocó, como si nunca hubiera pasado. No queda nada a medias. Lo único que se pierde es el gas.

view: la función lee el estado y no lo cambia; llamarla desde fuera **no genera transacción y no cuesta gas**. Es una pregunta, no una orden.

Validar primero, siempre: la guardia va en la primera línea, antes de tocar el estado.

¿Y `require`?

Existe, y lo van a ver en todos los tutoriales viejos: `require(condición, "texto");`. Hace lo mismo, pero el mensaje es un texto, cuesta más gas y no lleva datos estructurados. En esta clase `require` aparece **una sola vez**, en el paso 6, al final de `retirar()`, y ahí van a poder comparar las dos formas lado a lado. Para validar reglas de negocio, prefieran el error personalizado.

Qué hay que comprobar

Compilen y **vuelvan a desplegar**: el contrato de antes no tiene las funciones nuevas.

CUENTA ·	QUÉ LLAMAR	SALIDA ESPERADA
1	<code>estaDisponible()</code>	<code>true</code> , debajo del botón, sin transacción.
2	Cuenta 1 → <code>prestar()</code> , y <code>estaDisponible()</code>	Funciona; ahora <code>false</code> .
3	Cuenta 2 → <code>prestar()</code>	REVIERTE. La terminal marca la transacción como fallida y muestra el nombre del error, <code>EquipoOcupado</code> , con la dirección de la cuenta 1 adentro. Expandan el detalle de la transacción en la terminal y léanla .

4 Cuenta 2 → `devolver()`

REVIERTE con `NoLoTienesTu`. Sin esta guardia, cualquiera «devuelve» el equipo de otro y se lo lleva al segundo siguiente.

5 Cuenta 1 → `devolver()`, y
después cuenta 2 → `prestar()`

Las dos funcionan. El ciclo cierra.

QUÉ SE ROMPIÓ

Nada, con un clic: el contrato ya se defiende bien. Lo que está mal es el **diseño**: este contrato maneja **un** equipo. El laboratorio tiene cuarenta.

¿Vamos a desplegar cuarenta contratos? Cuarenta despliegues, cuarenta direcciones que alguien tiene que anotar, otro despliegue cada vez que entre un equipo nuevo, y cuarenta preguntas para saber qué hay disponible. Eso es el paso 3.

3

QUÉ HAY QUE ESCRIBIR · `pasos/paso-3.sol`El paso con más cambios: todas las funciones reciben un `id`.

- Se van las dos variables de estado del paso 1.
- Entra `struct Equipo` con tres campos: `string nombre`, `address prestadoA`, `bool existe`.
- `mapping(uint256 id => Equipo) public equipos;` y `uint256 public totalEquipos;`.
- Dos errores más: `EquipoNoRegistrado(uint256 id)` y `NombreVacio()`.
- `function registrar(string calldata nombre) public returns (uint256 id)`: rechaza el nombre vacío, incrementa `totalEquipos`, usa ese número como `id` y guarda el `Equipo`.
- `estaDisponible`, `prestar` y `devolver` pasan a recibir `uint256 id`, y las tres comprueban `existe`.

EL POR QUÉ DE CADA COSA, EN UNA LÍNEA

struct: un tipo que ustedes inventan, para que los datos que van juntos viajen juntos.

mapping: una tabla; usted da una llave y recibe un valor. Con **dos rarezas** que arruinan proyectos: (a) **no se puede recorrer** — no existe «dame todos los equipos», por eso llevamos `totalEquipos` a mano; (b) **toda llave existe desde siempre, llena de ceros** — pregunte por el equipo 7.000.000 y le contesta sin quejarse.

El campo existe: por la rareza (b). Es lo único que distingue «el equipo 99 no está registrado» de «el equipo 99 está libre». Sin él, el contrato contesta una mentira cómoda.

calldata: el texto se lee directo de los datos de la transacción, sin copiarlo a memoria. Es la opción más barata para un parámetro que la función no modifica.

Los id arrancan en 1: `totalEquipos++` va **antes** de asignar el `id`. Así «id 0» significa siempre «no existe», en un lenguaje donde todo arranca en cero.

Qué hay que comprobar

CUENTA · QUÉ LLAMAR	SALIDA ESPERADA
1 <code>registrar</code> con <code>"Proyector Epson"</code> (con comillas: es un <code>string</code>), y otra vez con <code>"Kit Arduino"</code> .	Las dos funcionan. <code>totalEquipos</code> → 2. Ojo : <code>registrar</code> devuelve el <code>id</code> , pero Remix no lo muestra, porque es una transacción. El valor de retorno de una transacción no lo puede leer nadie desde fuera. Eso lo arreglan los eventos , en el paso 6.
2 <code>equipos</code> con 1, y con 2.	Los tres campos: el nombre, <code>prestadoA</code> en la dirección cero, <code>existe: true</code> . Esta función de lectura no la escribieron: la regaló el <code>public</code> del <code>mapping</code> .
3 Cuenta 1 → <code>prestar(1)</code> . Cuenta 2 → <code>prestar(2)</code> .	Las dos funcionan y conviven : dos dueños distintos, un solo contrato.

4 Cuenta 2 → `prestar(1)` .

REVIERTE con `EquipoOcupado` y la cuenta 1 adentro. La defensa del paso 2 sigue viva para los cuarenta equipos: se escribió una sola vez.

5 `estaDisponible(99)` .

REVIERTE con `EquipoNoRegistrado(99)` . Fíjense en lo raro: es una función `view` , una simple pregunta, y **también** puede revertir. Sin el campo `existe` habría contestado `true` .

6 `prestar(99)` .

REVIERTE también con `EquipoNoRegistrado(99)` , **no** con `EquipoOcupado` : `prestar` llama a `estaDisponible` y el error de adentro sale hasta afuera tal cual. Los `revert` se propagan.

QUÉ SE ROMPIÓ · HÁGANLO USTEDES, CON LA CUENTA 3

Cambien a la **cuenta 3** (que es un estudiante cualquiera, no el laboratorio) y llamen a `registrar("Portatil que no existe")` .

Funciona. `totalEquipos` → 3 . Cualquiera puede meter inventario inventado en el registro del laboratorio, y el contrato le da las gracias.

Pregúntense: ¿quién debería poder registrar? Y la difícil: ¿*cómo* hace el contrato para saber que yo soy esa persona? No hay usuarios ni contraseñas, y no puede haberlos: todo lo que el contrato guarda es **público**, incluida la contraseña. Lo único que hay es: quién firmó. Eso es el paso 4.

4

QUÉ HAY QUE ESCRIBIR · `pasos/paso-4.sol`

Cuatro cosas, y una conversación que no es técnica

- `address public encargado;` y un error más: `SoloElEncargado(address quienLlamo)`.
- `constructor() { encargado = msg.sender; }.`
- `modifier soloEncargado() { if (msg.sender != encargado) revert SoloElEncargado(msg.sender); _; }.`
- Pegarle `soloEncargado` a `registrar`: va entre la visibilidad y el `returns`.
- `function recuperar(uint256 id) public soloEncargado`, que pone `prestadoA` en cero.

EL POR QUÉ DE CADA COSA, EN UNA LÍNEA

constructor : corre una sola vez en la vida del contrato, en el despliegue. Nunca más, no aparece como botón, no existe después. Lo que se grabe ahí queda grabado.

modifier : una condición que se repite, escrita una sola vez. El `_;` no es adorno: es el hueco donde se mete el cuerpo de la función. Lo que está antes del `_;` corre antes; por eso la validación va arriba.

recuperar() : alguien se graduó, perdió la billetera, o simplemente no devuelve. Sin esta función ese equipo queda prestado **para siempre**, y en la cadena «para siempre» es literal. Es la primera función que entra no porque algo se rompió, sino porque algo se puede *trancar* y no hay cómo destrabarlo.

Por qué prestar NO lleva el modificador: prestar lo puede hacer cualquiera; registrar, no. Decidir *qué* función lleva la guardia es el diseño; poner la guardia es lo fácil.

Qué hay que comprobar

CUENTA	QUÉ LLAMAR	SALIDA ESPERADA
1	Desplegar con la cuenta 1 y leer <code>encargado</code> .	La dirección de la cuenta 1. Nadie preguntó nada: lo decidió el hecho de que fue esa cuenta la que desplegó.
2	Cuenta 3 → <code>registrar("Portatil que no existe")</code> .	REVIERTE con <code>SoloElEncargado</code> y la dirección de la cuenta 3 adentro. El mismo clic que funcionaba en el paso 3 ahora se cae, y el error dice quién lo intentó.
3	Cuenta 1 → <code>registrar("Proyector Epson")</code> .	Funciona. <code>totalEquipos</code> → 1.
4	Cuenta 2 → <code>prestar(1)</code> .	Funciona: <code>prestar</code> no lleva el modificador, a propósito.
5	Cuenta 1 → <code>recuperar(1)</code> , y leer <code>equipos(1)</code> .	<code>prestadoA</code> vuelve a la dirección cero sin que la cuenta 2 haya hecho nada .
6	Cuenta 2 → <code>recuperar(1)</code> .	REVIERTE con <code>SoloElEncargado</code> .

QUÉ SE ROMPIÓ · NO ES TÉCNICO, Y ES LO MÁS IMPORTANTE DEL PASO

Acabamos de arreglar un problema y de crear uno más grande: le dimos a **una sola dirección** el poder de quitarle el equipo a cualquiera. Y miren el contrato: **no hay forma de cambiar de encargado**.

Si esa llave se pierde, nadie registra un equipo nunca más y nadie recupera nada, jamás. Si se la roban, el inventario es del ladrón. Y no se puede arreglar editando el contrato, porque ya está desplegado.

Lo grave no es tener un dueño: casi todo contrato tiene uno. Lo grave es no darse cuenta de que lo hay. Cuando lean un contrato ajeno, la primera pregunta es: *¿quién manda aquí, y qué pasa si esa llave se pierde?*

Lo que falta —poder traspasar el cargo— ya está escrito y auditado: se llama `Ownable`, de OpenZeppelin. Lo usan en la próxima sesión. Hoy se escribe a mano para que cuando lo usen sepan qué hay adentro.

Y falta algo obvio: nadie dijo hasta cuándo. Eso es el paso 5.

5

QUÉ HAY QUE ESCRIBIR · `pasos/paso-5.sol`

Un campo, una constante, un parámetro y dos consultas

- En el `struct`, un campo nuevo **antes** de `existe: uint64 vence;`.
- `uint64 public constant PLAZO_MAXIMO = 14 days;` y el error `PlazoInvalido(uint64 maximo)`.
- `prestar` **pasa a ser** `prestar(uint256 id, uint64 dias): calcula plazo = dias * 1 days`, rechaza `dias == 0` y `plazo > PLAZO_MAXIMO`, y fija `vence = uint64(block.timestamp) + plazo`.
- `devolver` y `recuperar` tienen que poner `vence = 0` además de `prestadoA`. **Es el olvido típico:** deja basura en el estado.
- `estaVencido(id)` y `diasDeMora(id)`, las dos `view`.

EL POR QUÉ DE CADA COSA, EN UNA LÍNEA

`block.timestamp`: el contrato **no tiene reloj**; no puede preguntar la hora ni llamar a un servidor. Lo único que sabe del tiempo es la hora que *el bloque dice que es*, en segundos, sin decimales. **Dos avisos que van siempre con esa línea:** (a) quien produce el bloque la puede mover unos segundos — para «catorce días» da igual, para sortear algo **jamás**; (b) es un `uint256`, por eso el `uint64(...)`.

La suma no hay que protegerla: si desbordara, Solidity 0.8 revierte sola. Antes de 2021 había que escribirlo a mano y ahí se perdió mucha plata.

`constant`: no ocupa estado, va grabada en el código. Y `14 days` es solo azúcar: el compilador lo convierte a segundos.

La división de `diasDeMora`: en Solidity **no hay decimales**. No hay `float`, no hay coma. La división **trunca**: un día y veintidós horas de mora son *un* día. No es un error del lenguaje: es una decisión que les toca tomar a propósito.

Por qué `uint64` y no `uint256` para `vence`: `address` (20 bytes) + `uint64` (8) + `bool` (1) = 29, y caben en **una** ranura de 32. Con `uint256` harían falta dos ranuras y cada préstamo costaría más. Ordenar los campos de un `struct` ahorra dinero real.

Qué hay que comprobar

CUENTA · QUÉ LLAMAR	SALIDA ESPERADA
1 Desplegar con cuenta 1, <code>registrar("Proyector Epson")</code> , y leer <code>PLAZO_MAXIMO</code> .	<code>1209600</code> . Son $14 \times 86\,400$: catorce días en segundos.
2 Cuenta 2 → <code>prestar(1, 2)</code> , y leer <code>equipos(1)</code> .	Funciona. <code>vence</code> es un número de diez cifras: el reloj de Unix, segundos desde el 1.º de enero de 1970. Es el único formato de fecha que hay acá.
3 <code>estaVencido(1)</code> y <code>diasDeMora(1)</code> .	<code>false</code> y <code>0</code> .

- | | | |
|---|--|---|
| 4 | Cuenta 1 → <code>devolver(1)</code> . | REVIERTE con <code>NoLoTienesTu</code> : el equipo es de la cuenta 2. |
| 5 | Cuenta 2 → <code>prestar(1, 0)</code> . | REVIERTE con <code>PlazoInvalido(1209600)</code> . El error no dice «mal»: dice cuál es el máximo. |
| 6 | Cuenta 2 → <code>prestar(1, 15)</code> , y después <code>prestar(1, 14)</code> . | El de 15 revierte; el de 14 entra . El borde exacto sí vale, y hay una prueba escrita solo para ese borde. |

AQUÍ SE ROMPE LA HERRAMIENTA, NO EL CONTRATO · Y ES IMPORTANTE QUE LO SEPAN

Van a querer ver la mora. Prestan el equipo por un día, esperan... y no pasa nada. **En la Remix VM el reloj no avanza solo**, y en una red de prueba pública como Sepolia tampoco se puede adelantar: el tiempo es el que es. No hay botón de «mañana».

Su contrato no está mal. Esta es la razón por la que todo contrato con plazos se prueba en una **red local propia**: es la única donde ustedes mandan el reloj. Cómo se hace está en la sección 11, y cómo se verifica sin Remix, en la 10.

QUÉ SE ROMPIÓ

`estaVencido(1)` dice `true` . El contrato **sabe** quién está en mora. ¿Y ahora qué? Nada. No puede ir a buscar el equipo, no puede llamar a nadie, no puede bloquear un carné.

Un contrato no tiene manos y no tiene teléfono. Lo único que puede hacer en el mundo es **mover plata**. Entonces, si estar en mora tiene que costar algo, tiene que costar plata, y la plata tiene que estar adentro antes. Eso es el paso 6.

6

QUÉ HAY QUE ESCRIBIR · `pasos/paso-6.sol`

El paso más largo. Hágalo en cuatro tramos y prueben entre cada uno.

1. **El dinero entra:** `uint256 public immutable deposito;` , el `constructor(uint256 deposito_)` que lo fija, la palabra `payable` en `prestar` , y la validación `if (msg.value != deposito) revert DepositoIncorrecto(deposito, msg.value);` .
2. **El dinero se anota:** `mapping(address quien => uint256 wei_) public saldo;` y, en `devolver` , calcular `bool aTiempo = block.timestamp <= equipos[id].vence;` y acreditar `saldo[aTiempo ? msg.sender : encargado] += deposito;` .
3. **El dinero se cobra:** `function retirar() public` , con el orden de las dos líneas finales **exacto**.
4. **Lo que queda:** los tres eventos y sus `emit` ; `quienLoTiene(id)` ; `recuperar` que ahora acredita el depósito al encargado; y `receive()` / `fallback()` .

EL POR QUÉ DE CADA COSA, EN UNA LÍNEA

Para qué todo esto: un contrato no tiene manos. Lo único que puede hacer en el mundo real es **tener plata y decidir si la devuelve**. Eso es lo único que lo hace distinto de un formulario con una base de datos: en el formulario la regla la aplica una persona que puede decidir no aplicarla; acá la regla y la plata viven en el mismo sitio y la regla se cumple sola.

payable : el interruptor que permite que entre ether. **Sin esa palabra la función rechaza cualquier envío:** no lo ignora, revierte. `msg.value` es cuánto venía en el sobre.

Monto exacto, ni un wei más: aceptar de más obligaría a devolver cambio, y devolverle plata a un desconocido es justo la operación delicada. Rechazar es gratis.

immutable : se fija al desplegar y no hay función que lo cambie. Más barato de leer que una variable normal.

El mapping saldo y el PATRÓN DE RETIRO: `devolver` **no envía** el dinero: solo lo anota. El contrato queda debiendo, y quien quiere cobrar llama a `retirar()` . Dos razones: (a) si el que recibe es un contrato que rechaza el envío, se caería la operación entera, y uno solo bloquearía el sistema para todos; (b) y esta es la grave: **enviar ether le entrega el control de la ejecución al que recibe**. Su código corre en medio del suyo y puede volver a entrar y cobrar otra vez. Así se perdieron 3,6 millones de ether en 2016.

El orden de las dos últimas líneas de `retirar()` no es estilo: el saldo en cero **ANTES** de enviar. Si el que recibe vuelve a entrar, ya no hay nada que cobrar. Invertir esas dos líneas vacía el contrato.

El `require(ok, "el envio fallo")` del final: la única vez en toda la clase. Compárenlo con los errores personalizados de arriba: mensaje de texto, más gas, sin datos. Para una condición técnica interna sirve; para una regla de negocio, no.

Eventos: no cambian el estado, escriben en el registro de la transacción. Cuestan mucho menos que guardar lo mismo y son de donde una interfaz o una auditoría saca la historia. Con una condición que sorprende: **un contrato no puede leer sus propios eventos**. Son para afuera. (Y resuelven lo del paso 3: el `id` que `registrar` devolvía y nadie podía ver.)

indexed en un evento: marca un parámetro para que se pueda **filtrar** desde fuera («dame todos los Prestado del equipo 1»). Hay un límite de tres por evento.

receive() y fallback() : nunca se llaman a propósito. **receive** corre si alguien manda ether sin decir para qué; **fallback**, si alguien llama a una función que no existe (nombre mal escrito, ABI viejo). Acá las dos **rechazan**, y es una decisión: plata que entra sin saber de quién ni para qué equipo es plata que después nadie sabe de quién era.

Qué hay que comprobar

CUENTA · QUÉ LLAMAR	SALIDA ESPERADA
1 Cuenta 1 → Deploy con <code>deposito_ = 1000000000000000000</code> . Leer <code>deposito</code> .	El mismo número. Son 0,1 ETH en wei (diecisiete ceros). Acá todo se cuenta en wei, la unidad más chica: un cero de menos y el depósito es una décima parte, sin que nada avise.
2 Cuenta 1 → <code>registrar("Proyector Epson")</code> . Expandir el detalle de la transacción en la terminal y buscar los logs.	El evento <code>EquipoRegistrado</code> con el <code>id</code> <code>1</code> y el nombre. Ahí está el <code>id</code> que en el paso 3 no se podía ver, y no costó una variable de estado.
3 Cuenta 2 → <code>prestar(1, 2)</code> con el campo <code>VALUE</code> en <code>0</code> .	REVIERTE con <code>DepositoIncorrecto</code> y los dos números adentro: <code>1000000000000000000</code> esperado, <code>0</code> recibido. El error no dice que se equivocaron: dice en cuánto .
4 Poner <code>VALUE</code> en <code>1000000000000000000</code> con la unidad en Wei (o <code>0.1</code> con la unidad en Ether) y repetir.	Funciona. En los <code>logs</code> , el evento <code>Prestado</code> con el <code>vence</code> . Si Remix les muestra el saldo de la cuenta, bajó un poco más de 0,1 ETH: 0,1 del depósito y el resto es gas. El gas no vuelve nunca , pase lo que pase con la transacción.
5 Cuenta 2 → <code>prestar(1, 2)</code> otra vez con el depósito correcto.	REVIERTE con <code>EquipoOcupado</code> y el depósito no se pierde : al revertir, el ether vuelve solo. Solo se pierde el gas.
6 Cuenta 2 → <code>devolver(1)</code> (a tiempo). Leer <code>saldo</code> con la dirección de la cuenta 2.	<code>1000000000000000000</code> . Y la billetera de la cuenta 2 no subió . El contrato dice que le debe 0,1 ETH y todavía no le pagó: no está roto, es el patrón de retiro . En los <code>logs</code> , <code>Devuelto</code> con <code>aTiempo: true</code> .
7 Cuenta 2 → <code>retirar()</code> , y volver a leer <code>saldo(cuenta 2)</code> .	Ahora sí llega el dinero. <code>saldo</code> → <code>0</code> .
8 Cuenta 2 → <code>retirar()</code> otra vez. Y cuenta 3 → <code>retirar()</code> .	Las dos REVIERTEN con <code>SinSaldo</code> . Nadie cobra dos veces y nadie cobra lo que no le deben.
9 Mandar 1 wei al contrato sin llamar ninguna función : en el panel de la instancia desplegada, el campo de interacción de bajo nivel (el que recibe <code>calldata</code>) vacío , con <code>VALUE</code> en <code>1 Wei</code> , y enviar.	REVIERTE con <code>EnvioDirectoNoPermitido</code> : entró por <code>receive()</code> . Ese ether no tenía dueño ni equipo.

SI NO ENCUENTRAN DÓNDE SE VE EL SALDO DE LA CUENTA

Dónde muestra Remix el saldo de la cuenta y de la instancia desplegada cambia entre versiones. No pierdan tiempo buscándolo: todo lo importante se comprueba con `saldo(dirección)`, que es una función del contrato, más el hecho de que la cuenta pudo pagar el depósito. La frase que importa es «el contrato me debe y todavía no me pagó», y eso lo prueba `saldo()`.

QUÉ QUEDÓ ROTO · Y ESTA ES LA ÚNICA QUE NO SE ARREGLA

Llegaron al final y el contrato tiene un hueco grande: **el que devuelve un minuto tarde pierde los 0,1 ETH completos**, igual que el que devuelve tres semanas tarde. Y `diasDeMora`, que escribieron en el paso 5 y que cuenta los días exactos, **no se usa para nada** en el paso 6.

La regla que se programó **no es la regla que se quería**. Ese es el primer ejercicio de ampliación (sección 13), y no tiene solución publicada en ninguna parte.

VENIR ACÁ ANTES DE PREGUNTAR

[illegible]

El <code>string</code> no se guarda bien	Lo escribieron sin comillas en el campo del argumento.	Los <code>string</code> van con comillas dobles: <code>"Proyector Epson"</code> . Los números, sin comillas.
Cambié el código y el contrato sigue igual	Compilaron pero no volvieron a desplegar. El código de un contrato desplegado no se puede cambiar .	Compilar y desplegar. Esto no es una molestia de Remix: es cómo funciona la cadena. Un contrato desplegado es inmutable.

De Solidity · errores de compilación y de ejecución

LO QUE DICE	POR QUÉ PASA	CÓMO SE ARREGLA
La versión del compilador no coincide con el <code>pragma</code>	El <code>pragma solidity 0.8.34;</code> es exacto y el compilador seleccionado es otro.	Pongan el compilador en <code>0.8.34+commit.80d5c536</code> . No cambien el <code>pragma</code> : la versión del curso está congelada a propósito.
Falta el especificador de ubicación de datos para un parámetro	Un parámetro de tipo <code>string</code> , <code>bytes</code> o arreglo en una función externa necesita <code>calldata</code> o <code>memory</code> .	<code>string calldata nombre</code> . Para un parámetro que no se modifica, <code>calldata</code> es lo más barato.
Identificador no declarado	Casi siempre una mayúscula: <code>Equipos</code> por <code>equipos</code> , o un error declarado después de usarse en otro archivo.	Solidity distingue mayúsculas. Lean el número de línea del error: está exacto.
Tipos no convertibles / falta una conversión explícita	Mezclaron <code>uint256</code> con <code>uint64</code> . <code>block.timestamp</code> es <code>uint256</code> y <code>vence</code> es <code>uint64</code> .	Conversión explícita: <code>uint64(block.timestamp)</code> . Solidity no convierte solo cuando se puede perder información.
La transacción revierte y no sé por qué	Alguna guardia se activó. El error dice cuál .	Expandan el detalle de la transacción en la terminal y lean el nombre del error y sus argumentos . Los seis errores del contrato están en la tabla de abajo.
Esperaba <code>EquipoOcupado</code> y salió <code>EquipoNoRegistrado</code>	El <code>id</code> no existe. <code>prestar</code> llama primero a <code>estaDisponible</code> , que revierte si el equipo no está registrado.	Registren el equipo primero. Y recuerden: los <code>id</code> empiezan en 1 , no en 0 .
Esperaba <code>DepositoIncorrecto</code> y salió <code>PlazoInvalido</code>	El orden de las validaciones : <code>prestar</code> comprueba disponibilidad, después el plazo, y por último el depósito.	No es un error: es lo que significa «validar primero, siempre». Si pasan <code>dias = 0</code> , nunca llegan a mirar el dinero.
<code>diasDeMora</code> devuelve menos de lo que esperaba	La división entera trunca . No hay decimales en Solidity.	No es un error del lenguaje. Si necesitan precisión, trabajen en una unidad más chica (horas, o segundos) y decidan el redondeo a propósito.

`estaVencido` siempre da `false` aunque pasó el plazo

En la Remix VM **el reloj no avanza solo**, y en una red pública no se puede adelantar.

No es un error de su contrato. Secciones 10 y 11: o corren las pruebas, o mueven el reloj de una red local.

Los seis errores del contrato final, y qué significa cada uno

ERROR	QUIERE DECIR
<code>EquipoNoRegistrado(id)</code>	Ese <code>id</code> no existe en el inventario. Los <code>id</code> empiezan en 1.
<code>EquipoOcupado(loTiene)</code>	Está prestado, y la dirección que trae adentro es la de quien lo tiene.
<code>NoLoTienesTu()</code>	Están intentando devolver un equipo que no es suyo.
<code>SoloElEncargado(quienLlamo)</code>	Esa función es solo del encargado, y el error dice qué dirección lo intentó.
<code>PlazoInvalido(maximo)</code>	Pidieron 0 días o más del máximo. El número que trae es el máximo en segundos: 1209600.
<code>DepositoIncorrecto(esperado, recibido)</code>	El <i>VALUE</i> no era el depósito exacto. Trae los dos números para comparar.
<code>SinSaldo()</code>	El contrato no les debe nada. Ya cobraron, o devolvieron en mora.
<code>NombreVacio()</code>	Registraron con <code>""</code> .
<code>EnvioDirectoNoPermitido()</code>	Mandaron ether sin llamar una función, o llamaron a una que no existe.

10 Cómo correr las pruebas

25 PRUEBAS · 3 MINUTOS

El contrato final del paso 6 está también en el proyecto de Hardhat del curso, con **25 pruebas automáticas**. Sirven para dos cosas: **demostrar la mora**, que en Remix no se puede, y **comprobar que su versión se comporta igual** que la de referencia.

Hace falta Node.js 22 y haber hecho `npm install` una vez en `material/laboratorios-evm` (eso se hizo en la Sesión 5).

```
cd material/laboratorios-evm
npx hardhat test test/demo/PrestamoEquipos.test.js
```

Salida esperada: seis bloques de pruebas y **25 en verde, cero fallas**. Los bloques son: inventario (6), prestar (4), devolver a tiempo (5), **mora (4)**, recuperar a la fuerza (3) y dinero que llega solo (3).

LAS CUATRO PRUEBAS QUE VALE LA PENA LEER CON CALMA

- **Pasado el plazo, `estaVencido` es cierto y `diasDeMora` cuenta los días:** presta por 1 día, adelanta 4, y espera 3. Esto es lo que no se puede hacer en Remix.
- **La división entera trunca: 1,9 días de mora se cuentan como 1.** La prueba *mide* lo que la guía afirma.
- **Devolver en mora: el depósito queda para el laboratorio.** Y el equipo vuelve a estar disponible: el laboratorio no pierde el activo.
- **El borde: devolver en el último segundo del plazo cuenta como a tiempo.** Fija el `<=` de `devolver`. Si fuera `<`, alguien perdería 0,1 ETH por un segundo.

Abran `test/demo/PrestamoEquipos.test.js` y lean esas cuatro. No hace falta entender todavía cómo se escriben: hace falta ver que **una prueba dice en una línea qué comportamiento se exige**. En la Sesión 8 las van a escribir ustedes.

SI QUIEREN PROBAR SU PROPIO CÓDIGO

Las pruebas compilan lo que haya en `contracts/demo/PrestamoEquipos.sol`, que es la copia de referencia del paso 6. Para probar **lo que ustedes escribieron**, péguenlo ahí **en su copia local** del repositorio y vuelvan a correr. Si pasan las 25, el comportamiento es el correcto.

11 Cómo mover el reloj

LA ÚNICA FORMA DE VER LA MORA EN REMIX

La Remix VM no deja adelantar el reloj y una red pública tampoco. Pero una **red local propia** sí: ahí el reloj es suyo. Esto conecta Remix con una red de Hardhat que corre en su computador.

QUÉ HACER

SALIDA ESPERADA

1 En una terminal, dentro de <code>material/laboratorios-evm</code>: <pre>npx hardhat node</pre> Déjenla abierta: esa terminal es la red.	Imprime una lista de cuentas de prueba con sus claves privadas y queda escuchando. Datos de la red: chainId 31337, RPC http://127.0.0.1:8545. Esas llaves privadas son públicas y de juguete: están en la documentación de Hardhat, las tiene todo el mundo. Nunca les manden nada real.
2 En Remix, <i>ENVIRONMENT</i> → el entorno de desarrollo que conecta con un nodo de Hardhat. La documentación de Remix lo llama Hardhat Provider; en el desplegable puede aparecer dentro del grupo <i>Dev</i>. El nodo del punto 1 ya tiene que estar corriendo.	El selector <i>Account</i> cambia: ahora son las cuentas del nodo local, no las de la Remix VM. El contrato que tenían desplegado no viaja: hay que volver a desplegar.
3 Desplegar ahí el paso 6, <code>registrar</code> un equipo y <code>prestar(1, 1)</code> con el depósito.	<code>estaVencido(1) → false</code>, <code>diasDeMora(1) → 0</code>.
4 En una segunda terminal, en la misma carpeta: PowerShell: <pre>\$env:DIAS="3"; npx hardhat run scripts/demo/adelantar-reloj.js --network localhost</pre> bash: <pre>DIAS=3 npx hardhat run scripts/demo/adelantar-reloj.js --network localhost</pre>	Tres líneas: el bloque y la fecha antes, el bloque y la fecha después, y <pre>el reloj avanzó 3.00 días (259200 segundos)</pre> Si dice que no pudo mover el reloj, es que el nodo del punto 1 no está corriendo o falta <code>--network localhost</code>.
5 En Remix, volver a llamar <code>estaVencido(1)</code> y <code>diasDeMora(1)</code>.	<code>estaVencido → true</code>. <code>diasDeMora → 2</code> (tres días transcurridos menos un día de plazo).
6 <code>devolver(1)</code> con la cuenta que lo tiene, y leer <code>saldo</code> de esa cuenta y del <code>encargado</code>.	El de quien devolvió: <code>0</code>. El del encargado: <code>1000000000000000000</code>. El depósito se quedó en el laboratorio. En los <i>logs</i>, <code>Devuelto</code> con <code>aTiempo: false</code>.

LA IDEA QUE HAY QUE LLEVARSE DE ESTA SECCIÓN

Ustedes acaban de **mover el tiempo**. En una red local eso se puede, y por eso una red local sirve para probar. En una cadena pública no se puede, y **por eso una cadena pública sirve para algo**. Si el reloj de la red donde vive el contrato lo puede mover el que administra la red, el plazo no vale nada.

12 Preguntas de repaso

RESPONDER SIN VOLVER A LEER

Estas cinco no se contestan copiando una definición: hay que haber hecho los pasos. Escriban la respuesta antes de buscarla.

1

PASO 1 · LA ROTURA SIN ERROR

En el paso 1, la cuenta 2 se llevó un equipo que tenía la cuenta 1 y la transacción salió **bien**. Explique por qué es un error grave aunque no haya ningún mensaje de error, y diga qué clase de problema es ese: ¿de sintaxis, de lógica, o de reglas que nadie escribió?

2

PASO 3 · EL CAMPO `existe`

¿Por qué el `struct Equipo` necesita el campo `bool existe`, si el `mapping` ya contesta cuando se le pregunta por cualquier `id`? Diga exactamente qué contestaría `estaDisponible(99)` sin ese campo, y por qué esa respuesta es peor que un error.

3

El encargado se graduó y perdió la billetera. Diga **qué deja de funcionar exactamente** en el contrato del paso 6 (nombre las funciones), y por qué no se puede arreglar editando el código. Después diga qué le falta al contrato para que esto no fuera un problema.

4

En `retirar()`, el saldo se pone en cero **antes** de enviar el dinero. Explique qué podría hacer un contrato malicioso si esas dos líneas estuvieran al revés, y por qué el problema aparece justo en el momento de enviar ether y no antes.

5

Para el laboratorio de equipos de **nuestra** universidad, ¿este contrato hacía falta? Responda sí o no y defiéndalo en cinco líneas. Después describa **un cambio concreto en la situación** (no en el código) que daría vuelta su respuesta.

13 Ejercicios de ampliación

SIN SOLUCIÓN PUBLICADA

Los tres parten del paso 6. **No hay solución en ninguna parte**, ni en el repositorio público ni en el del docente: el valor está en pelearse con la decisión de diseño, no en llegar a un código concreto. Para cada uno, antes de escribir, contesten por escrito las tres preguntas que trae.

A

EL QUE ARREGLA EL HUECO QUE QUEDÓ ABIERTO

Cobrar la mora **por día**, en vez de quedarse con todo el depósito

Hoy el que devuelve un minuto tarde pierde los 0,1 ETH completos, igual que el que devuelve tres semanas tarde. Y `diasDeMora` existe desde el paso 5 y no se usa. Hagan que `devolver` cobre una multa por día de mora y acredite el resto a quien devolvió.

Antes de escribir, contesten:

1. La multa diaria, ¿se fija al desplegar como el depósito, o se calcula como una fracción de él? Defiendan la elección.
2. **No hay decimales.** Si la multa es «el 10 % del depósito por día», ¿cómo se calcula eso sin coma, y qué pasa con lo que se trunca? ¿Quién se queda con el resto?
3. ¿Qué pasa si la mora acumulada **pasa** el depósito? ¿Se queda en el depósito, o el contrato empieza a reclamar una deuda que no puede cobrar? Decidan, y digan por qué es una decisión y no una cuenta.

Cómo saben que avanzaron: cuando puedan explicar por qué su respuesta a la pregunta 3 es la correcta *para este caso*, aunque la otra también sea defendible.

B

EL QUE PARECE FÁCIL Y NO LO ES

Permitir **prórroga**: extender un préstamo antes de que se venza

Agreguen una función que le permita a quien tiene el equipo correr la fecha de vencimiento.

Antes de escribir, contesten:

1. ¿Quién puede pedir la prórroga: solo quien lo tiene, o también el encargado? ¿Y se puede pedir **estando ya en mora**? Si se puede, la multa deja de existir: cualquiera prorroga el día que vence. Si no se puede, ¿qué le pasa a alguien que se enfermó?
2. El `PLAZO_MAXIMO` es de 14 días. ¿La prórroga se suma sobre el vencimiento, o el total sigue sin poder pasar de 14 días desde el préstamo? Son dos contratos distintos.
3. ¿Cuesta algo prorrogar? Si es gratis, ¿qué impide prorrogar indefinidamente y quedarse con el equipo todo el semestre, cumpliendo las reglas?

Cómo saben que avanzaron: cuando encuentren solos que la pregunta 1 puede volver inútil toda la mora del paso 6.

Llevar **historial** de quién tuvo cada equipo

Hoy el contrato solo sabe quién tiene el equipo **ahora**. Hagan que se pueda saber quién lo tuvo antes.

Antes de escribir, contesten:

1. El `mapping` **no se puede recorrer**. ¿Van a guardar el historial en el estado (un arreglo por equipo, que crece y cuesta gas en cada préstamo) o les basta con los **eventos** que ya existen, que son casi gratis pero que **el contrato no puede leer**? Decidan según quién necesita el historial: ¿una pantalla, u otra función del contrato?
2. Si crece en el estado: ¿qué pasa con el gas de `prestar` en el préstamo número doscientos? ¿Y quién paga ese costo que sube?
3. **La pregunta que no es técnica.** «Quién tuvo cada equipo» es un dato que, cruzado con el horario de un curso, puede identificar a una persona. La cadena es **pública e inmutable**, y la Ley 1581 de 2012 le da a toda persona derecho a rectificar y a pedir la supresión de sus datos. ¿Se puede cumplir ese derecho sobre algo que no se puede borrar? ¿Qué guardarían en su lugar?

Cómo saben que avanzaron: cuando puedan defender, por escrito y en cinco líneas, que la respuesta correcta a la pregunta 3 puede ser **no guardar el historial en la cadena**.

14 Dónde está todo, y lo que hay que verificar

CIERRE

QUÉ	DÓNDE
Las seis versiones del contrato	<code>material/clase-solidity-paso-a-paso/pasos/paso-1.sol ... paso-6.sol</code>
Copia probada de la versión final	<code>material/laboratorios- evm/contracts/demo/PrestamoEquipos.sol</code>
Las 25 pruebas	<code>material/laboratorios- evm/test/demo/PrestamoEquipos.test.js</code>
El script para mover el reloj	<code>material/laboratorios-evm/scripts/demo/adelantar-reloj.js</code>
Remix IDE	<code>https://app.remix.live</code>
Documentación de Remix · entornos y despliegue	<code>https://remix-ide.readthedocs.io/en/latest/run.html https://remix- ide.readthedocs.io/en/latest/create_deploy.html</code>
Versiones congeladas del semestre	<code>material/laboratorios-evm/versiones.md</code>

SI LA INTERFAZ DE REMIX NO COINCIDE CON ESTA GUÍA

Remix cambia de vez en cuando y hay cosas que esta guía describe **a propósito** de forma genérica, porque el nombre exacto varía entre versiones: el nombre del entorno que conecta con un nodo de Hardhat, el nombre del *fork* de cada Remix VM, dónde se muestra el saldo de una cuenta, y cómo se llama el campo de interacción de bajo nivel para mandar ether sin llamar una función.

Cuando eso pase, **busquen el nombre de la acción, no la posición del botón**, y confirmen en la documentación de arriba. Si encuentran que algo de esta guía quedó viejo, **avísenlo por el canal del curso con el enlace de la documentación**: se corrige el material. Reportar una fuente que quedó vieja es trabajo del curso, no una queja.

LAS TRES FRASES QUE DEBERÍAN QUEDAR

1. **En Solidity nadie dice quién es: lo demuestra firmando.** No hay usuarios ni contraseñas, porque todo lo que el contrato guarda es público. Solo hay `msg.sender`.
2. **Un contrato no tiene manos.** Lo único que puede hacer en el mundo es tener plata y decidir si la devuelve. Si una regla tiene que costar algo, tiene que costar plata.
3. **La habilidad que vale no es escribir Solidity.** Es saber cuándo *no* usar una blockchain y poder defender esa decisión. Para este laboratorio, la respuesta honesta es que no hacía falta.